

Three Roads — Stack Decision

Constraints, three options, a pressure-test, and the call

The four constraints

Budget: Free only — no paid tiers, no credit card that could quietly start billing.

Skill level: Complete beginner — new to HTML/CSS itself, not just frameworks.

What the portfolio needs to do: Match the existing sitemap/content map: a hero with one claim, one case study led by real screenshots and long-form reading, an Evaluation & Artifacts section linking to a code repo, About, Contact — all static content, no live product embedded yet.

Display needs: Real image galleries (screenshots), long-form reading (the case study), and a code-repo link. No embedded live demo required today.

Dynamic/backend, right now?: Not yet. A real login + database is wanted eventually, but nothing on the current content map needs it today.

Three options, simplest to most powerful

	A — Static site	B — Framework, static now	C — Full-stack from day one
How you'd build	Plain HTML/CSS, one file, hand-edited or AI-assisted	Next.js (or similar), exported/deployed as a static site for now	Next.js frontend + Supabase for real auth and a Postgres database, wired in from the start
Free host	Netlify or GitHub Pages	Vercel or Netlify free tier	Vercel free tier + Supabase free tier
Needs a backend?	No	Not yet — but the same platform adds serverless functions/auth later without switching hosts	Yes, immediately — real accounts and a real database exist from the first deploy
Real trade-off	Zero learning curve, ships today. When real login/database is actually needed, it's a genuine addition later, not a five-minute toggle.	Real learning curve now (React/Next basics, build tooling, git) for a total beginner — but the upgrade path to real backend later is clean.	Most future-proof, but also the most to learn and maintain right now — env vars, schema, auth flows — for content that doesn't need any of it yet.

Pressure-testing the front-runner: Option A

What breaks if I pick the simplest?

Nothing breaks today — the current content map is 100% static (text, real screenshots, a repo link). It only 'breaks' the day real login/database is actually needed, and at that point it's an honest migration, not a patch.

What do I maintain if I pick the most powerful?

Environment variables, a live database schema, authentication flows, and a build pipeline — for content that today is just a hero, a case study, and a contact link. That's real maintenance load with zero current payoff.

Can I finish in two weeks, brand new to HTML/CSS?

With Option A, yes — it's the same file already built and already deployed. With Option C, a total beginner debugging auth and database errors inside a two-week window is a genuine risk of not finishing at all.

Does it show my work the way it needs to be shown?

Yes — real screenshots, long-form case-study text, and a repo link are all plain static content. None of that needs a backend to display correctly.

The call

Chosen: Option A — static HTML, deployed free on Netlify.

I'm new to this, still learning HTML/CSS itself, and everything the portfolio actually needs to show right now — screenshots, a case-study write-up, a repo link — is static content. Picking the full-stack option now would mean learning a database and a login system before I've even finished learning HTML, for a page that doesn't use either one yet. Solving a problem I don't have yet would just cost me the two-week deadline I do have.

Why not B: the framework route is useful later, but it adds real learning overhead right now (React, a build step, more git to deal with) for zero benefit today — the content map doesn't need anything a framework gives that plain HTML doesn't already cover.

Why not C: it fails my own 'can I maintain this' test right now — a real database and real login system are more moving parts than I can debug confidently as a beginner, and none of it is used by anything on the page yet.

Can I maintain this? Yes — it's one file, plain HTML/CSS, already live. Nothing is running, nothing needs patching, nothing can break except the free host itself.

Does it show my work well? Yes — the case study leads with real screenshots and reads as a long-form write-up, exactly what the content map asked for.

The honest backend answer: not yet. When a real login/database feature is actually needed, that's the point to move toward Option B's framework path — not before.